Final Project Report
Natural Language Processing Course

A Persian Question Answering System Based on Retrieval-Augmented Generation

UkAsk

Author:
Amirhossein Abolhassani

Spring 2026

Contents

1 Introduction **1**

2 Overall System Architecture **1**

3 Document Extraction and Cleaning **2**
3.1 Faulty Character Mapping Table and Reliance on OCR 2
3.2 Reversal of the Lam-Alef Sequence 2
3.3 Header and Footer Removal 2
3.4 Table Extraction and the Decision to Disable It 2

4 Document Ingestion Pipeline **3**

5 Question Answering Pipeline **3**
5.1 Retrieval 3
5.2 Query Relevance Gating 3
5.3 Generation 4

6 Evaluation Method **4**
6.1 Evaluation Set and Split 4
6.2 Retrieval Metrics 4
6.3 Whole-System Evaluation 4

7 Results **5**
7.1 Retrieval on the Development Set 5
7.2 Retrieval on the Test Set 6
7.3 System Accuracy on the Test Set 7

8 Discussion **8**
8.1 Component-Level versus System-Level Evaluation 8
8.2 Re-ranking as the Next Step 9

9 Conclusion **9**

References **10**

Appendices **10**

A Reciprocal Rank Fusion **10**

B Retrieval Metric Formulas **11**

C System Prompt **12**

1 Introduction

The goal of this project is to design and implement a Persian question answering system that answers the questions of students and faculty members about the educational regulations of Shahid Bahonar University of Kerman. The system's knowledge source consists of seven official PDF documents, a mixture of text-based documents and scanned documents.

The overall approach is a Retrieval-Augmented Generation (RAG) architecture. In this architecture, instead of having a large language model answer directly from its own memory, the most relevant parts of the source documents are first retrieved, and the language model then generates the answer solely on the basis of those parts. This has two main advantages. First, the answers remain grounded in the official source and can be cited by article number and page. Second, when there is insufficient information, the system refuses to answer rather than fabricating an incorrect one.

The final output is a complete web application in which the user can enter a question in Persian and receive a grounded answer together with a list of sources (article number, page number, and similarity score). This report focuses on the practical findings of the project, in particular the difficulties of extracting text from real administrative documents and the interesting discrepancy between retrieval-level evaluation and whole-system evaluation.

2 Overall System Architecture

The system architecture is divided into two independent pipelines. The document processing and preparation pipeline runs offline, that is, once, or whenever the source documents change. The question answering pipeline runs online, once for each user question. Figure 1 shows an overview of these two pipelines.

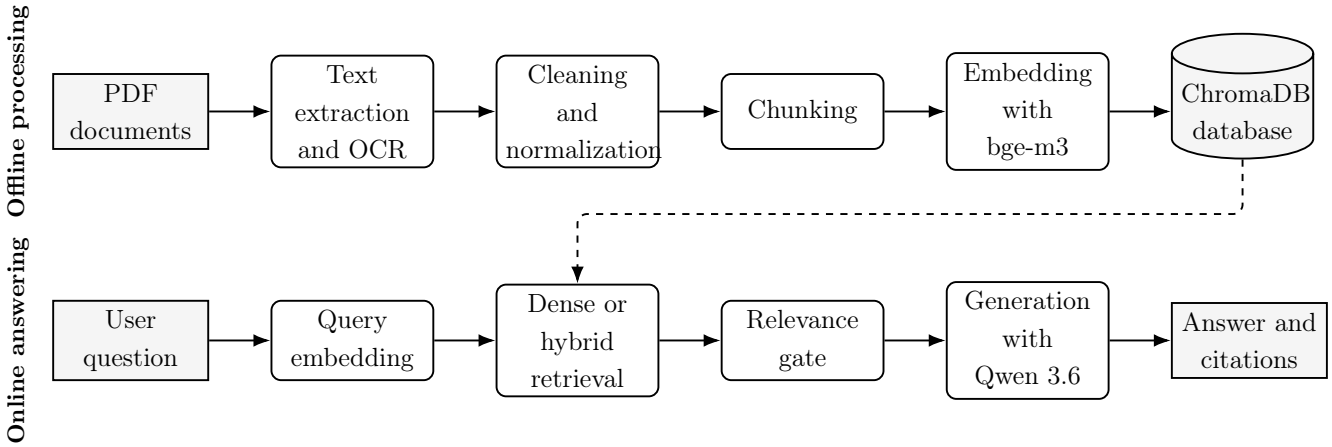


Figure 1: Overall architecture of the UkAsk system

3 Document Extraction and Cleaning

The most important unexpected finding of this project appeared at the very first stage, namely extracting text from the PDF documents. These seven real administrative documents had several different text-extraction defects, none of which was visible when viewing the document normally, and all of which were revealed only when the text was extracted programmatically. [1]

3.1 Faulty Character Mapping Table and Reliance on OCR

In one of the documents, the embedded font had a faulty character mapping table (ToUnicode CMap), meaning that some portions were extracted as blank spaces or as completely wrong characters. This was confirmed by directly inspecting the font's internal mapping table. Since the document was displayed correctly on screen but its text was extracted incorrectly, OCR was more reliable than text extraction for this particular case. For this reason, the system can fall back to OCR both automatically at the level of each page (when the native text is insufficient) and by manual override for this entire document.

3.2 Reversal of the Lam-Alef Sequence

A different defect in another document caused the extremely common lam-alef sequence to be extracted in reversed order. After analyzing the document, it became clear that this stems from the drawing order of the glyphs. The main challenge is that a corrupted "la→al" and a genuine, correct "al" (as in the words "sāl", "māl", and "hāl") are textually indistinguishable after extraction. For this reason, the problem was solved with a conservative correction: a word is corrected only when its alternative reading is both common and meaningfully more common than the extracted reading. Ambiguous or low-signal cases are deliberately left uncorrected so that no new errors are introduced into the text.

3.3 Header and Footer Removal

Repeated headers and page numbers were detected and removed automatically. The method is to find lines that are repeated on a large number of pages. Two explicit safeguards were included for this. The first safeguard is for short documents, where any line is trivially repeated across a small number of pages and might be removed by mistake. The second safeguard is for genuine regulatory content: every article of the regulations begins with the phrase "Article N", which is a repeating structure that must be preserved.

3.4 Table Extraction and the Decision to Disable It

Table detection and extraction (based on grid lines and text position, with conversion to Markdown) was implemented and evaluated. However, late in development an unresolved bug in right-to-left text reversal, specific to the extraction of table cells, was discovered. Given its negative

[1] These problems were solved by the solution proposed by the AI.

effect on the evaluation metrics (the table content could not be attributed to a specific article number), this feature was deliberately disabled in the current version, and instead of being removed entirely, it was documented as a disabled feature of the system that can be developed further.

4 Document Ingestion Pipeline

After extraction and cleaning, the text is ready to be converted into a searchable vector representation. The document ingestion pipeline consists of the following stages.

1. **Chunking.** The text was split into overlapping token windows of 300 to 500 tokens. It should be noted that the overlap between chunks prevents a concept from being cut at the boundary between two chunks.

2. **Metadata.** For each chunk, the page number, the document title, and any article number that appears in that chunk were recorded. It was important to note that a chunk may span several articles, or may continue an article without repeating its number.

3. **Embedding.** Each chunk was converted to a vector with the `BAAI/bge-m3` model [1]. This model was chosen for its multilingual and Persian retrieval quality [2] and was run locally in order to avoid the per-query call cost and the dependence on an external service.

4. **Storage.** The vectors were stored in the ChromaDB vector database. Cosine similarity was also used as the similarity measure for this vector database.

5 Question Answering Pipeline

5.1 Retrieval

The user's question is first converted to a vector with the same `bge-m3` model. Two retrieval approaches were implemented. The baseline approach is dense retrieval based on cosine similarity. The second approach is hybrid retrieval, which combines dense search with `BM25` keyword search via Reciprocal Rank Fusion (see Appendix A).

5.2 Query Relevance Gating

Before any generation, a gating layer compares the similarity of the top-ranked retrieved document against a calibrated threshold. If no result is sufficiently relevant, the system rejects the request directly, without any call to the model. The threshold value was tuned on the dataset (a value close to 0.5). This gate both prevents answer generation for irrelevant questions and keeps the model-call cost for such questions at zero.

5.3 Generation

Answer generation was carried out through the `GapGPT` service (an OpenAI-API-compatible proxy, with the model `Qwen 3.6`). The system prompt instructs the model to answer only from the retrieved documents, to cite the article number and page, and, when the documents are insufficient, to use a predefined refusal phrase (the full prompt is listed in Appendix C). The final output is structured and consists of the answer text together with a list of sources (page, article numbers, and similarity score).

6 Evaluation Method

6.1 Evaluation Set and Split

A set of one hundred questions provided by the course teaching assistant (91 fact-based questions and 9 reasoning questions, each with a reference answer, a source article, and a difficulty level) served as the basis of the evaluation. This set was split into a twenty-question development set and an eighty-question test set. The split was stratified by question type and difficulty so that the share of the small subset of reasoning questions was preserved proportionally and the balance was the same in both parts. All hyperparameter tuning was done only on the development set, and the test set was not used until the hyperparameters were finalized.

6.2 Retrieval Metrics

The retrieval metrics (Hit Rate at k[1], Mean Reciprocal Rank[2], Precision at k[3], and Mean Average Precision[4]) were computed by matching the reference article number of each question against the article metadata of the retrieved chunks. The formulas of all four metrics are given in Appendix B.

There is a limitation in the provided dataset, and consequently in metrics such as Precision@k and Mean Average Precision: the values of these metrics depend on knowing the true number of relevant articles for each question. This is because, in these 7 documents, each question may be answered by several different passages. In such a situation, the system may answer the question correctly while not having retrieved the stated article. As a result, the metrics mentioned are less reliable. Hit rate and mean reciprocal rank do not depend on this approximation and were considered the primary, more defensible metrics. Precision at k and MAP were kept as secondary evidence, meaningful mainly for the relative comparison of dense versus hybrid retrieval and not as a measure of the system's performance on its own.

6.3 Whole-System Evaluation

The accuracy of the RAG system as a whole was measured using an LLM-as-judge approach with the `gpt-4o` model. Each generated answer was compared with the reference answer solely on the

[1] HitRate@k
[2] MRR
[3] Precision@k
[4] MAP

basis of factual content and classified into three categories: correct, partial, and incorrect. The judge model was also given a prompt asking it to state the output format as well as its reasoning for each answer.

7 Results

7.1 Retrieval on the Development Set

Figures 2 and 3 show the trend of the retrieval metrics as a function of k on the development set for the hybrid and dense approaches. In the production configuration ($k = 5$), the hit rate was 0.65 for dense and 0.60 for hybrid, and the mean reciprocal rank of the two remained close to each other (about 0.44 to 0.47). The advantage of the hybrid approach did not appear at $k = 5$, but became clearly visible at larger values of k: the hit rate at $k = 10$ was 0.75 versus 0.65 for dense, and at $k = 15$ it was 0.80 versus 0.75. The key point is that, in both approaches, the mean reciprocal rank remained almost constant as k increased. This means that the additional chunks retrieved at larger k lie deep in the ranking and are not promoted upward. It should be noted that this is a problem of ranking quality, distinct from a problem in retrieval itself.

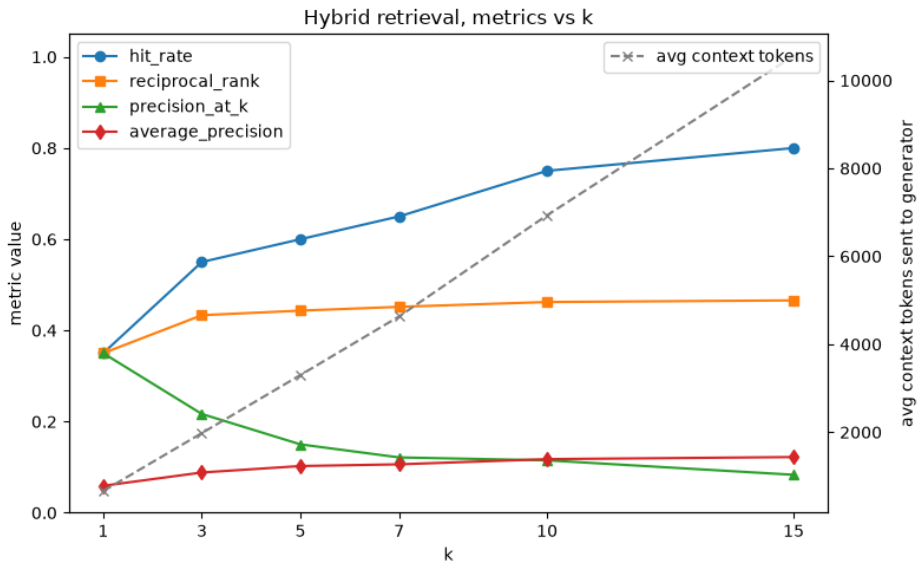


Figure 2: Hybrid retrieval, metrics as a function of k (development set)

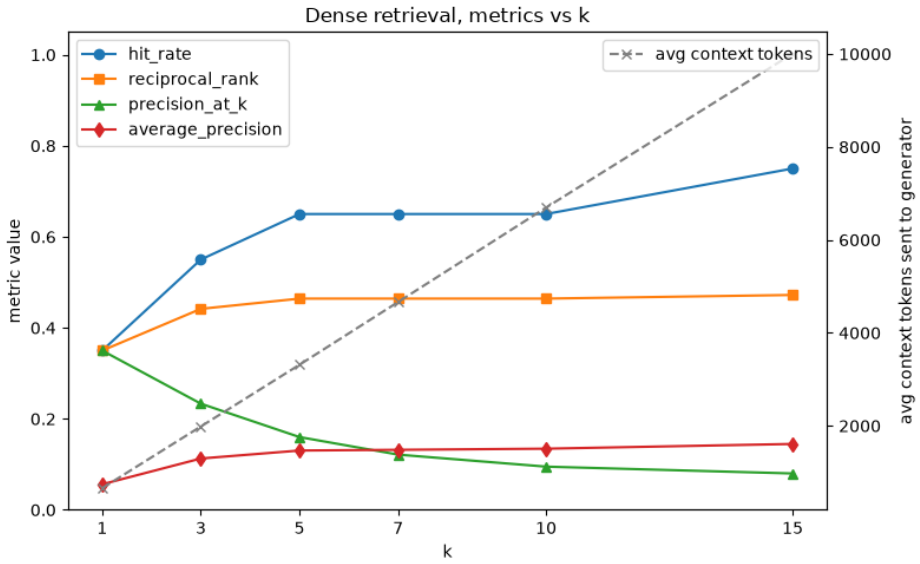


Figure 3: Dense retrieval, metrics as a function of k (development set)

7.2 Retrieval on the Test Set

Table 1 shows the retrieval metrics on the eighty-question test set. In contrast to the development set, which, because of its small size (twenty questions), showed dense slightly ahead at $k = 5$, on the larger test set the hybrid approach is ahead on all four metrics.

Table 1: Overall retrieval metrics on the test set

Metric	**Dense**	**Hybrid**
Hit Rate	0.425	0.512
Mean Reciprocal Rank	0.260	0.321
Precision@k	0.127	0.143
Mean Average Precision	0.094	0.112

Table 2 shows the breakdown of the metrics by question type. On fact-based questions, the hybrid approach is clearly better. On reasoning questions, the hit rate of the two approaches is equal, but the mean reciprocal rank of hybrid is lower. This is despite the fact that the reasoning subset is very small, and these results cannot be relied upon without reservation.

Table 2: Retrieval metrics by question type (test set)

Question type	**Method**	**Hit**	**MRR**	**Precision@k**	**MAP**
Fact-based	Dense	0.425	0.265	0.132	0.101
Fact-based	Hybrid	0.521	0.342	0.148	0.119
Reasoning	Dense	0.429	0.207	0.086	0.016
Reasoning	Hybrid	0.429	0.105	0.086	0.040

Table 3 shows the breakdown by difficulty level. Medium-level questions have the highest retrieval quality. The hybrid approach shows a clear advantage at the easy and medium levels. However, on hard questions, the hit rate of both approaches at this level is only 0.176, and the hybrid approach does not compensate for this weakness.

Table 3: Retrieval metrics by difficulty (test set)

Difficulty	**Method**	**Hit**	**MRR**	**Precision@k**	**MAP**
Easy	Dense	0.483	0.264	0.152	0.114
Easy	Hybrid	0.552	0.363	0.166	0.129
Medium	Dense	0.500	0.344	0.153	0.120
Medium	Hybrid	0.647	0.425	0.176	0.145
Hard	Dense	0.176	0.085	0.035	0.007
Hard	Hybrid	0.176	0.043	0.035	0.016

7.3 System Accuracy on the Test Set

Figure 4 and Table 4 show the overall accuracy of the system on the test set using the language model as judge. The hybrid approach reduced the rate of clearly incorrect answers by more than half compared with dense (from 8 to 3 out of 80), which is the best result in the entire evaluation. This is despite the fact that the retrieval metrics at ($k = 5$) on the development set had not shown a clear advantage for the hybrid approach. This is an interesting divergence between retrieval-level and system-level evaluation, which we address in Section 8.1.

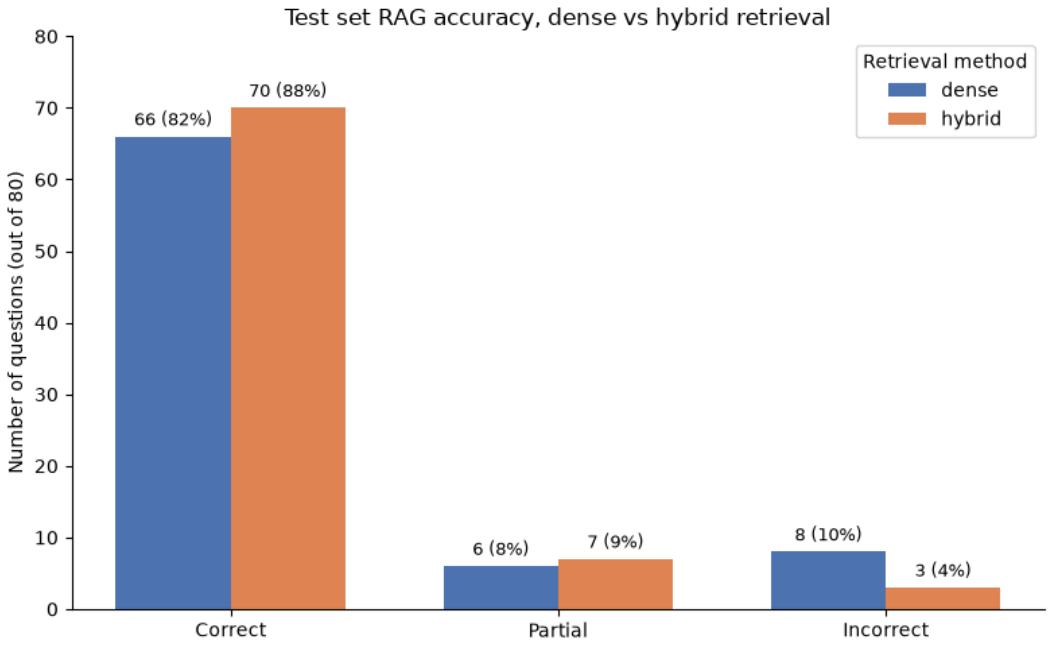


Figure 4: End-to-end accuracy on the test set, dense approach compared with hybrid approach

Table 4: System accuracy on the test set ($n = 80$)

Judgment	Dense	Hybrid
Correct	66 (82%)	70 (88%)
Partial	6 (8%)	7 (9%)
Incorrect	8 (10%)	3 (4%)

Figure 5 shows the system-level judgments broken down by difficulty. Here the most important contrast of the entire evaluation can be seen. As shown in Table 3, retrieval on hard questions was very weak (hit rate of about 0.176), yet at the answer level almost all of those same hard questions were answered correctly (15 correct out of 17 for the dense approach, and all 17 hard questions correct for the hybrid approach). This confirms the finding of Section 6 that a miss on the article number does not mean the answer is definitely wrong. Rather, in most cases an alternative source is retrieved.

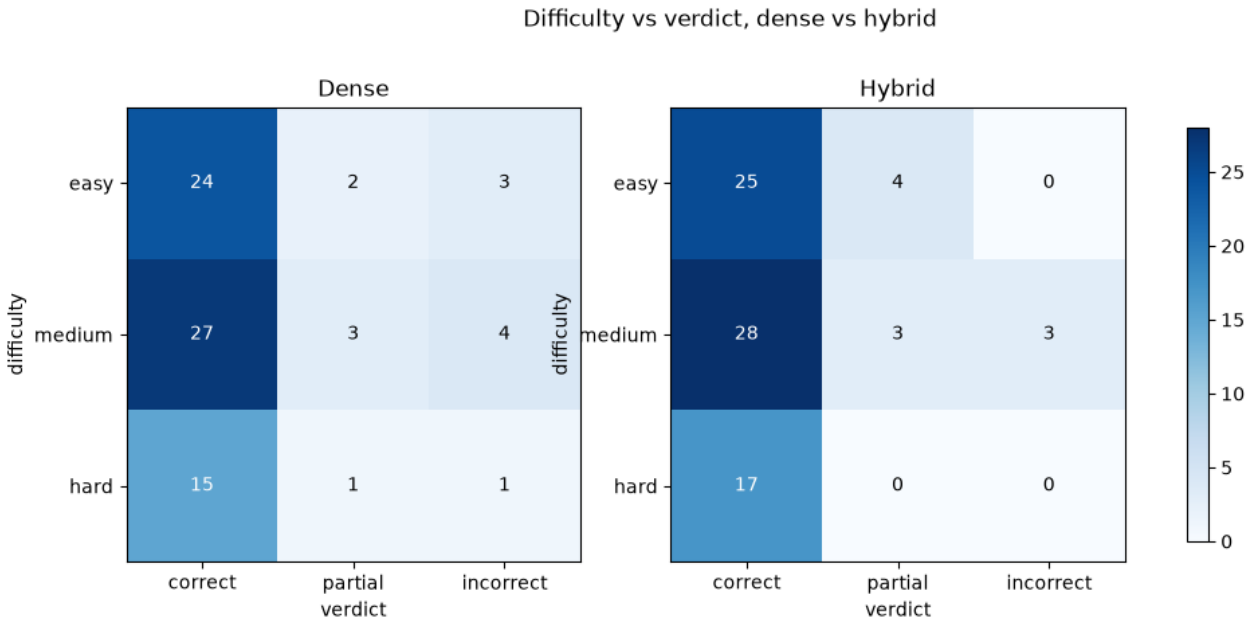


Figure 5: System-level judgments by difficulty, dense approach versus hybrid approach

8 Discussion

8.1 Component-Level versus System-Level Evaluation

Retrieval-level evaluation and system-level evaluation gave two different pictures of the value of the hybrid retrieval approach. The retrieval metrics alone indicated that the hybrid approach was not worth adopting, whereas the overall system accuracy showed a good and meaningful value. This is attributed in part to the model's ability to use relevant content from any point of the documents given to it, and not only the top-ranked chunk. My conclusion from this observation is that, for a RAG system, system-level evaluation is the final decision criterion, and relying solely on retrieval

metrics can be somewhat misleading.

8.2 Re-ranking as the Next Step

The pattern in Figure 2, in which the mean reciprocal rank stays constant while the hit rate increases as k grows, clearly points toward re-ranking. This targets exactly the problem of finding the document but failing to improve the rank metric. This capability was not implemented in the current version and has been identified as future work.

9 Conclusion

In this project, a Persian question answering system for the educational regulations of the university was implemented, covering everything from document extraction and cleaning to retrieval and answer generation.

References

[1] Chen, J. et al. (2024). BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation.

[2] FaMTEB: Massive Text Embedding Benchmark in Persian Language. arXiv:2502.11571 (2025).

[3] Chroma. ChromaDB: the open-source embedding database. `https://www.trychroma.com`

[4] Cormack, G. V., Clarke, C. L. A., Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR.

[5] Robertson, S., Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond.

[6] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

Appendix A Reciprocal Rank Fusion[1]

Hybrid retrieval produces two independent ranked lists, one from dense search and one from `BM25`. The dense search score is a cosine similarity in a bounded range, whereas the `BM25` score is a lexical score with a completely different distribution. For this reason, directly adding these two raw scores is meaningless. Reciprocal Rank Fusion sidesteps this problem because, instead of the raw score, it uses only the rank of each document in each list, and is therefore insensitive to the scale of the scores. For each document d, the final score is given by the following formula.

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{\kappa + \mathrm{rank}_r(d)} \tag{A.1}$$

The meaning of each symbol in this formula is as follows. d is a document, which in this project is a chunk of the corpus, whose final score we want to compute. R is the set of input ranked lists. In this project there are two lists, one from dense search and one from `BM25`. r is a specific ranked list among the members of R, and the summation sign indicates that the document's contributions from all of these lists are added together. $\mathrm{rank}_r(d)$ is the one-based rank of document d in list r, that is, rank 1 for the top result, rank 2 for the next result, and so on. If document d does not appear in list r at all, the contribution of that list to the sum is taken to be zero. κ is a positive smoothing constant whose common value is 60; its role is to control how sharply the contribution falls off with rank and to prevent the first few ranks from dominating too much.

The key point is the presence of the rank in the denominator. Since a smaller rank means a better result, the denominator is smaller and hence the contribution of that list is larger; the lower a document appears in a list, the more rapidly its contribution decreases. The role of κ is to soften this decrease; with a small κ, the differences between the top ranks become very pronounced, and with a large κ the list becomes smoother and rank differences have less effect.

[1] Generated by AI

The final score of each document is the sum of its contributions from both lists. A document that ranks well in both lists receives the highest cumulative score, but a document that ranks highly in only one list also receives a meaningful contribution and is not discarded altogether. This very property causes matches that are strong both semantically (from dense search) and lexically (from `BM25`) to rise to the top of the final list.

To make this clearer, suppose document d has rank 2 in the dense list and rank 5 in the `BM25` list. With $\kappa = 60$, its score is computed as follows.

$$\mathrm{RRF}(d) = \frac{1}{60+2} + \frac{1}{60+5} = \frac{1}{62} + \frac{1}{65} \approx 0.0315 \tag{A.2}$$

A document that is absent from one of the two lists receives only one of these two terms and therefore has a lower score.

Appendix B Retrieval Metric Formulas

In the following formulas, Q is the set of questions, k is the number of top results, and rank_q is the rank of the first relevant chunk for question q. If no relevant chunk is retrieved, the contribution of that question to MRR is taken to be zero.

Hit Rate@k is the proportion of questions for which at least one relevant chunk appears in their top k results.

$$\text{Hit Rate@}k = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{1}[\mathrm{rank}_q \leq k] \tag{B.1}$$

MRR is the mean of the reciprocal rank of the first relevant chunk and is sensitive to ranking quality.

$$\mathrm{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\mathrm{rank}_q} \tag{B.2}$$

Precision@k is the proportion of relevant chunks among the top k results, where $r_q(k)$ is the number of relevant chunks in the top k results of question q.

$$\text{Precision@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{r_q(k)}{k} \tag{B.3}$$

MAP is the mean of Average Precision over all questions, where $P(i)$ is the precision at cutoff i, $\mathrm{rel}(i)$ equals one if the chunk at rank i is relevant and zero otherwise, and R_q is the total number of relevant chunks for question q.

$$\mathrm{AP}_q = \frac{1}{R_q} \sum_{i=1}^{k} P(i)\,\mathrm{rel}(i) \tag{B.4}$$

$$\mathrm{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \mathrm{AP}_q \tag{B.5}$$

Appendix C System Prompt

This instruction is kept as a constant string variable, and its structure is given below. (The original prompt is written in Persian; it is rendered here in English translation.)

SYSTEM_PROMPT =

You are an assistant that answers university questions on the basis of the regulations. Answer only on the basis of the texts provided in the "Documents" section, but you may perform only a little interpolation to improve the quality and reasonableness of the answer. Write the answer at a medium length, neither too brief nor too explanatory; usually one to six sentences are enough, unless the user asks for a full explanation in the question, or unless the question genuinely requires more explanation. In the answer, refer to the article number and the page of the relevant document, without any introduction or extra explanation. If the answer to the question does not exist in the provided documents, return exactly this sentence and write nothing else:
"The answer to this question was not found in the available documents."
If you notice writing problems in the text, you do not necessarily have to return the sentence exactly as it is; you may also edit it and interpolate yourself.